

PRACTICAL_WEEK-09

Task 1: Implement a file copy utility using low-level Linux file I/O system calls (`open()`, `read()`, `write()`, `lseek()`, `close()`). Compare its performance with standard library functions (`fopen()`, `fread()`, `fwrite()`).

```
Last login: Mon Sep 21 13:31:26 on ttys000
amrutha@Amruthas-MacBook-Air ~ % cd ~/OSSP
amrutha@Amruthas-MacBook-Air OSSP % ls
assets                quote_flowchart.txt    wait_flowchart.txt
bin                   README.md              week3_task3_flowchart.txt
docs                  scripts                week3_task4_flowchart.txt
double_quote_flowchart.txt skill5_task1_flowchart.txt week5_task1_flowchart.txt
escape_flowchart.txt  skill5_task2_flowchart.txt week5_task2_flowchart.txt
File1.txt             skill6_task1_flowchart.txt week6_task1_flowchart.txt
File2.txt             skill6_task2_flowchart.txt week6_task2_flowchart.txt
include               skill7_task1_flowchart.txt week7_task1_flowchart.txt
LICENSE               skill7_task2_flowchart.txt week7_task2_flowchart.txt
Makefile              skill8_task1_flowchart.txt week8_task1_flowchart.txt
obj                   skill8_task2_flowchart.txt week8_task2_flowchart.txt
parser_flowchart.txt  src
process_flowchart.txt tests
amrutha@Amruthas-MacBook-Air OSSP %
```

Fig 1: Navigating to the OSSP project directory for Practical 9

```
UW PICO 5.09                                     File: src/week9_t

    size_t n;

    clock_gettime(CLOCK_MONOTONIC, &start);

    while ((n = fread(buffer, 1, BUFFER_SIZE, source)) > 0)
    {
        fwrite(buffer, 1, n, dest);
    }

    clock_gettime(CLOCK_MONOTONIC, &end);

    fclose(source);
    fclose(dest);

    double stdio_time = elapsed_time(start, end);

    printf("File copied successfully using both methods.\n");
    printf("Low-Level I/O Time : %.9f seconds\n", low_level_time);
    printf("Standard I/O Time : %.9f seconds\n", stdio_time);

    return 0;
}
```

Fig 2: C program comparing file copying using low-level system calls and standard library I/O functions

```

amrutha@Amruthas-MacBook-Air OSSP % echo "Operating Systems Practical 9 File Copy Test" > source.txt
echo "This file will be copied using low-level I/O and standard library I/O." >> source.txt
echo "We are comparing open/read/write with fopen/fread/fwrite." >> source.txt
amrutha@Amruthas-MacBook-Air OSSP % cat source.txt
Operating Systems Practical 9 File Copy Test
This file will be copied using low-level I/O and standard library I/O.
We are comparing open/read/write with fopen/fread/fwrite.
amrutha@Amruthas-MacBook-Air OSSP %

```

Fig 3: Creation of the source file used for file-copy performance testing

```

amrutha@Amruthas-MacBook-Air OSSP % gcc src/week9_task1.c -o bin/week9_task1
amrutha@Amruthas-MacBook-Air OSSP % ./bin/week9_task1
File copied successfully using both methods.
Low-Level I/O Time : 0.000366000 seconds
Standard I/O Time : 0.000021000 seconds
amrutha@Amruthas-MacBook-Air OSSP %

```

Fig 4: Execution output comparing low-level file I/O and standard library file-copy performance

```

amrutha@Amruthas-MacBook-Air OSSP % cat lowlevel_copy.txt
Operating Systems Practical 9 File Copy Test
This file will be copied using low-level I/O and standard library I/O.
We are comparing open/read/write with fopen/fread/fwrite.
amrutha@Amruthas-MacBook-Air OSSP % cat stdio_copy.txt
Operating Systems Practical 9 File Copy Test
This file will be copied using low-level I/O and standard library I/O.
We are comparing open/read/write with fopen/fread/fwrite.
amrutha@Amruthas-MacBook-Air OSSP % cmp source.txt lowlevel_copy.txt
cmp source.txt stdio_copy.txt
amrutha@Amruthas-MacBook-Air OSSP %

```

Fig 5: Verification that both copied files match the original source file

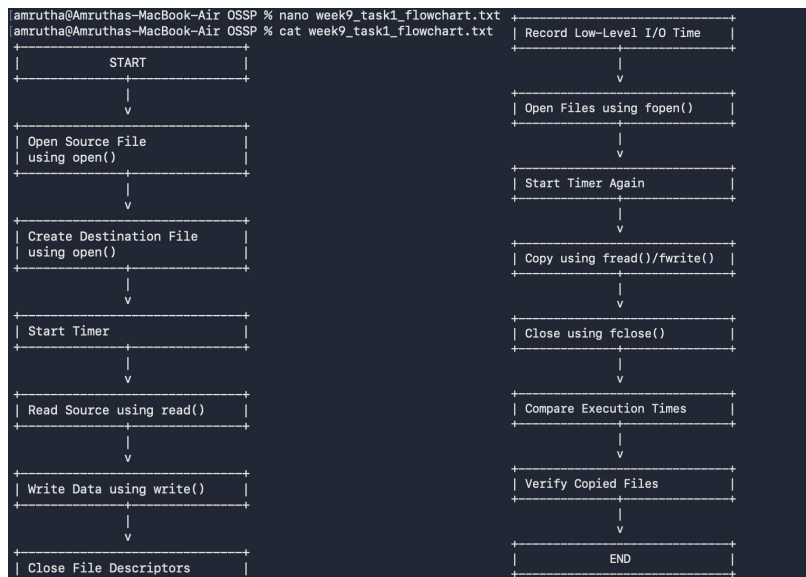


Fig 6: Flowchart for file copying using low-level and standard library I/O methods

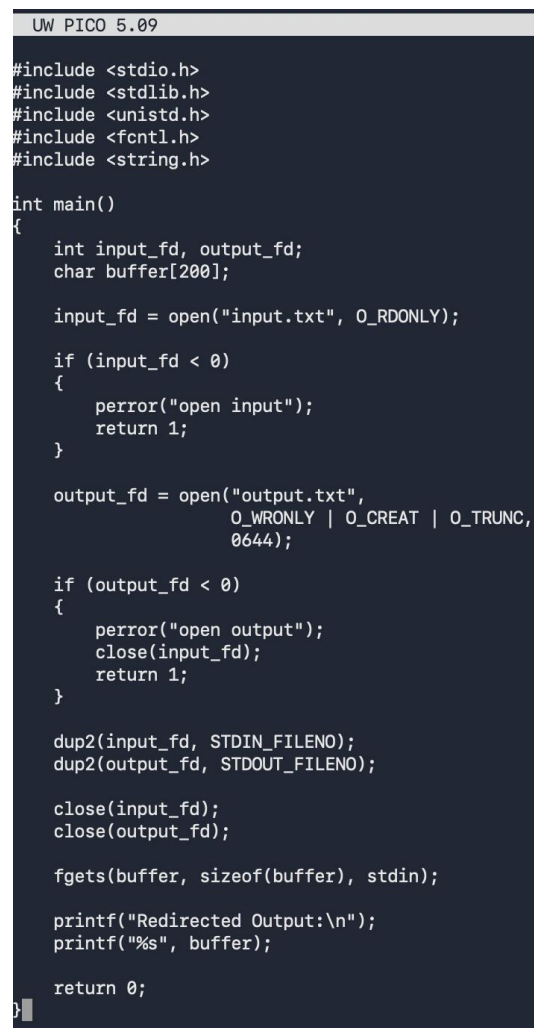
Observation:

The source file was copied successfully using both low-level system calls and standard library functions. The execution times of both methods were measured and compared, and the copied files matched the original file.

Result:

File copying using `open()`, `read()`, `write()`, and `close()` as well as `fopen()`, `fread()`, and `fwrite()` was implemented successfully. The performance of both approaches was compared, and the correctness of the copied files was verified.

Task 2: Develop a program that redirects standard input and output using file descriptors and `dup2()`. Demonstrate how shells implement I/O redirection.



```
UW PICO 5.09
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>

int main()
{
    int input_fd, output_fd;
    char buffer[200];

    input_fd = open("input.txt", O_RDONLY);

    if (input_fd < 0)
    {
        perror("open input");
        return 1;
    }

    output_fd = open("output.txt",
                    O_WRONLY | O_CREAT | O_TRUNC,
                    0644);

    if (output_fd < 0)
    {
        perror("open output");
        close(input_fd);
        return 1;
    }

    dup2(input_fd, STDIN_FILENO);
    dup2(output_fd, STDOUT_FILENO);

    close(input_fd);
    close(output_fd);

    fgets(buffer, sizeof(buffer), stdin);

    printf("Redirected Output:\n");
    printf("%s", buffer);

    return 0;
}
```

Fig 7: C program demonstrating standard input and output redirection using `dup2()`

```

amrutha@Amruthas-MacBook-Air OSSP % echo "Operating Systems I/O Redirection Test" > input.txt
amrutha@Amruthas-MacBook-Air OSSP % cat input.txt
Operating Systems I/O Redirection Test
amrutha@Amruthas-MacBook-Air OSSP % █

```

Fig 8: Creation of the input file used for standard input redirection

```

amrutha@Amruthas-MacBook-Air OSSP % gcc src/week9_task2.c -o bin/week9_task2
amrutha@Amruthas-MacBook-Air OSSP % ./bin/week9_task2
amrutha@Amruthas-MacBook-Air OSSP % cat output.txt
Redirected Output:
Operating Systems I/O Redirection Test
amrutha@Amruthas-MacBook-Air OSSP % █

```

Fig 9: Output written to a file after redirecting standard input and output using dup2()

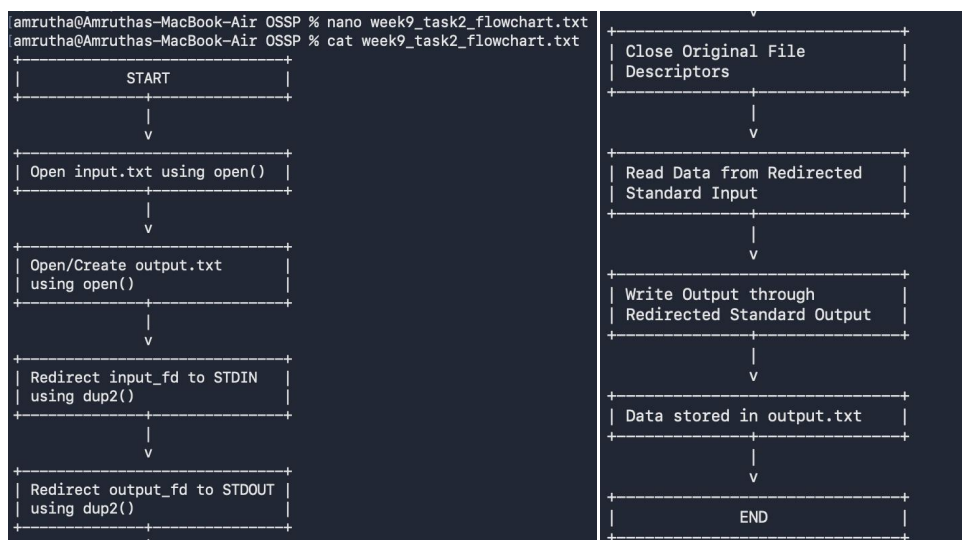


Fig 10: Flowchart for standard input and output redirection using dup2()

Observation:

Standard input was redirected from input.txt, and standard output was redirected to output.txt using dup2(). The program successfully demonstrated file-descriptor based I/O redirection similar to shell redirection.

Result:

Standard input and output redirection using open() and dup2() was implemented successfully. Data was read from the input file and written to the output file through redirected standard file descriptors.